Chapter 1
Modeling Market Returns

In an environment subject to risk and randomness, any result or prediction has to rely on a specified statistical model. This chapter reviews basic Gaussian modeling of the returns of risky assets using Brownian motion and geometric Brownian motion. We also provide a statistical benchmarking of these Gaussian-based models against actual market returns.

1.1 Random Walks **3**
1.2 Market Returns **10**
1.3 Geometric Brownian Motion **13**
1.4 Distribution of Market Returns **21**
1.5 Gram–Charlier Expansions **30**
Exercises **35**

1.1 Random Walks

Our approach to the modeling of the price of risky assets is based on the use of random walks indexed by discrete or continuous time. We consider the random walk $(S_n)_{n\geqslant 0}$, also called the *Bernoulli* random walk, and defined by letting $S_0 := 0$ and

$$S_n := \sum_{k=1}^{n} X_k = X_1 + \cdots + X_n, \qquad n \geqslant 1,$$

where the increments $(X_k)_{k\geqslant 1}$ form a sequence of *independent and identically distributed* (*i.i.d.*) Bernoulli random variables, with distribution

$$\begin{cases} \mathbb{P}(X_k = +1) = p, \\ \mathbb{P}(X_k = -1) = q, \qquad k \geqslant 1, \end{cases}$$

with $p + q = 1$. In other words, the random walk $(S_n)_{n\geqslant 0}$ can only evolve by going up or down by one unit within the finite state space $\{0, 1, \ldots, S\}$. We have

$$\mathbb{P}(S_{n+1} = k+1 \mid S_n = k) = p \text{ and } \mathbb{P}(S_{n+1} = k-1 \mid S_n = k) = q, \quad k \in \mathbb{Z}.$$

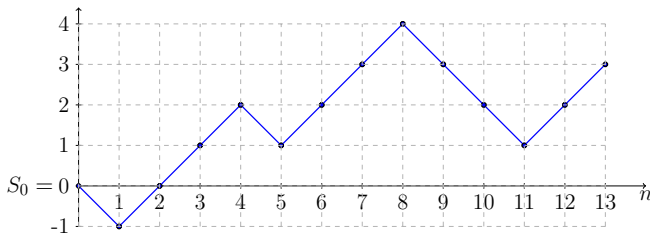


Fig. 1.1: Sample path of the random walk $(S_n)_{n\geqslant 0}$.

Proposition 1.1. *The mean value of S_n can be expressed as*

$$\mathbb{E}[S_n \mid S_0 = 0] = \mathbb{E}\left[\sum_{k=1}^{n} X_k\right] = \sum_{k=1}^{n} \mathbb{E}\left[X_k\right] = (2p-1)n = (p-q)n,$$

and its variance can be computed as

$$\mathrm{Var}\left[S_n \mid S_0 = 0\right] = \mathrm{Var}\left[\sum_{k=1}^{n} X_k\right] = \sum_{k=1}^{n} \mathrm{Var}\left[X_k\right] = 4npq.$$

We note that in an even number of time steps, $(S_n)_{n\in\mathbb{N}}$ can only reach an even state in $\mathbb{Z}$ after starting from 0. Similarly, in an odd number of time steps, $(S_n)_{n\in\mathbb{N}}$ can only reach an odd state in $\mathbb{Z}$ after starting from 0. In Proposition 1.2 we compute the distribution of S_{2n} for all $n \geqslant 1$.

Proposition 1.2. *The probability distribution of S_{2n}, $n \geqslant 1$, is given by*

$$\mathbb{P}(S_{2n} = 2k \mid S_0 = 0) = \binom{2n}{n+k} p^{n+k} q^{n-k}, \quad -n \leqslant k \leqslant n. \tag{1.1}$$

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

Figure 1.2 provides an illustration of the result of Proposition 1.2 by enumerating the $120 = \binom{10}{7} = \binom{10}{3}$ possible paths corresponding to $n = 5$ and $k = 2$, which all have the same probability $p^{n+k}q^{n-k} = p^7q^3$ of occurring.

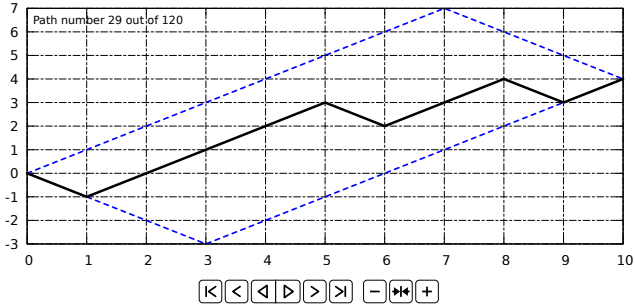


Fig. 1.2: Graph of $120 = \binom{10}{7} = \binom{10}{3}$ paths with $n = 5$ and $k = 2$.*

Brownian Motion

The modeling of random assets in finance is mainly based on stochastic processes, which are families $(X_t)_{t\in I}$ of random variables indexed by a time interval I. Brownian motion is a fundamental example of a stochastic process.

Brown (1828) observed the movement of pollen particles as described in "A brief account of microscopical observations made in the months of June, July and August, 1827, on the particles contained in the pollen of plants; and on the general existence of active molecules in organic and inorganic bodies." Phil. Mag. 4, 161-173, 1828.

* Animated figure (works in Acrobat Reader).

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

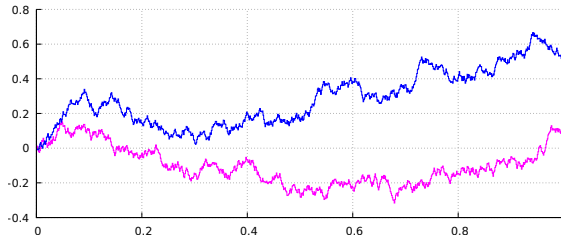


Fig. 1.3: Two sample paths of one-dimensional Brownian motion.

Einstein (1905) received his 1921 Nobel Prize in part for investigations on the theory of Brownian motion: "... in 1905 Einstein founded a kinetic theory to account for this movement", presentation speech by S. Arrhenius, Chairman of the Nobel Committee, Dec. 10, 1922.

Bachelier (1900) used Brownian motion for the modeling of stock prices in his PhD thesis "Théorie de la spéculation", Annales Scientifiques de l'Ecole Normale Supérieure 3 (17): 21-86, 1900.

Wiener (1923) is credited, among other fundamental contributions, for the mathematical foundation of Brownian motion, published in 1923. In particular he constructed the Wiener space and Wiener measure on $\mathcal{C}_0([0,1])$ (the space of continuous functions from $[0,1]$ to $\mathbb{R}$ vanishing at 0).

Itô (1944) constructed the Itô integral with respect to Brownian motion, and the stochastic calculus with respect to Brownian motion, which laid the foundation for the development of calculus for random processes, see Itô (1951) "On stochastic differential equations", *in* Memoirs of the American Mathematical Society.

From random walks to Brownian motion

Letting $T > 0$, we split the time interval $[0,T]$ into N subintervals

$$(t_{k-1}, t_k], \qquad k = 1, 2, \ldots, N,$$

of same length $\Delta t = T/N$, where $t_k = kT/N$, $k = 0, 1, \ldots N$, and N is "large".

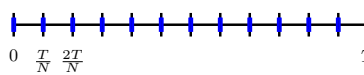


Fig. 1.4: Discrete time scale.

We will informally regard Brownian motion as a random walk over infinitesimal time intervals of length Δt, whose increments

$$\Delta B_t := B_{t+\Delta t} - B_t$$

over the time interval $[t, t+\Delta t]$ will be approximated by the Bernoulli random variable

$$\Delta B_t \approx \pm\sqrt{\Delta t} \tag{1.2}$$

with equal probabilities $(1/2, 1/2)$. Figure 1.5 presents a simulation of Brownian motion as a random walk with $\Delta t = 0.1$.

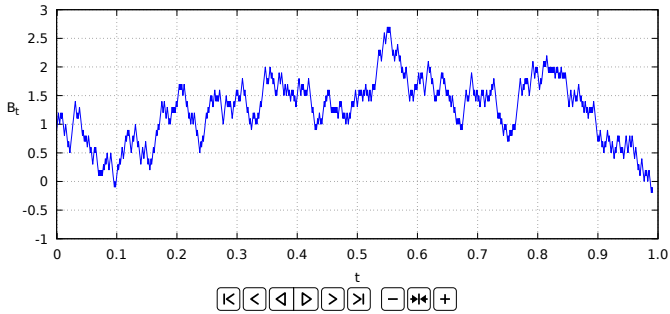


Fig. 1.5: Construction of Brownian motion as a random walk.*

We note that

$$\mathbb{E}[\Delta B_t] = \frac{1}{2}\sqrt{\Delta t} - \frac{1}{2}\sqrt{\Delta t} = 0,$$

and

$$\mathrm{Var}[\Delta B_t] = \mathbb{E}\left[(\Delta B_t)^2\right] = \frac{1}{2}\Delta t + \frac{1}{2}\Delta t = \Delta t.$$

Gaussianity of Brownian motion

Let

$$X_k := \pm\sqrt{T} = \pm\sqrt{N\Delta t}, \qquad k = 1, 2, \ldots N,$$

denote a sequence of symmetric Bernoulli random variables taking values in $\{-\sqrt{T}, \sqrt{T}\}$ with equal probabilities $(1/2, 1/2)$, such that $\mathbb{E}[X_k] = 0$ and variance $\mathrm{Var}[X_k] = T$, $k = 1, \ldots, N$. Since

* The animation works in Acrobat Reader on the entire pdf file.

$$\Delta B_{t_k} = \pm\sqrt{\Delta t} = \pm\sqrt{\frac{T}{N}} = \frac{X_k}{\sqrt{N}}, \qquad k = 1, 2, \ldots N,$$

we can write

$$B_T \simeq \sum_{k=1}^{N} \Delta B_{t_k} = \frac{X_1 + X_2 + \cdots + X_N}{\sqrt{N}},$$

hence by the central limit theorem we recover the fact that B_T has the centered Gaussian distribution $\mathcal{N}(0, T)$ with variance T, cf. point 4 of Definition 1.4 of Brownian motion, and the illustration given in Figure 1.6.

The following code plots a set of simulated Brownian motion paths according to the construction of increments provided in (1.2).

```
beta=.5; nsim=100; N=1000; t=0:N; dt=1.0/N; dev.new(width=16,height=7);
X=matrix((dt)^beta*(rbinom( nsim * N, 1,.5)-.5)*2, nsim, N)
X=cbind(rep(0, nsim), t(apply(X, 1, cumsum))); H=hist(X[,N],plot=FALSE);
layout(matrix(c(1,2), nrow =1, byrow = TRUE));par(mar=c(2,2,2,0), oma = c(2, 2, 2, 2))
plot(t*dt, X[1, ], type="l", ylim=c(-2,2), col=0, xaxs="i", las=1, cex.axis=1.6)
for (i in 1:nsim){lines(t*dt, X[i, ], type = "l", ylim = c(-2, 2), col = i)}
lines(t*dt,sqrt(t*dt),lty=1,col="red",lwd=3);lines(t*dt,-sqrt(t*dt), lty=1, col="red",lwd=3)
lines(t*dt,0*t, lty=1, lwd=2); for (i in 1:nsim){points(.999, X[i,N], pch=1, lwd = 5, col = i)};
x=seq(-2,2, length=100); px=dnorm(x);par(mar = c(2,2,2,2))
plot(NULL , xlab="", ylab="", xlim = c(0, max(px,H$density)), ylim = c(-2,2),axes=F)
rect(0, H$breaks[1:(length(H$breaks) - 1)], col=rainbow(20,start=.08,end=.6), H$density,
  H$breaks[2:length(H$breaks)]); lines(px,x, lty=1, lwd=2)
```

Listing 1.1: R code - Brownian motion simulation.

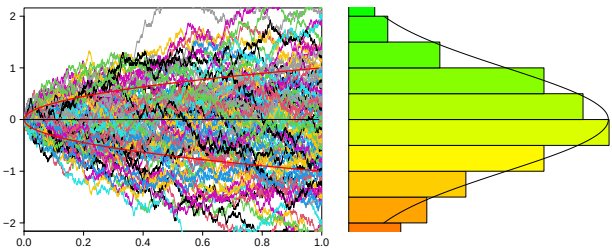


Fig. 1.6: Statistics of one-dimensional Brownian paths *vs.* Gaussian distribution.

```
import numpy as np, matplotlib.pyplot as plt;
nsim, N= 100, 1000; t= np.arange(0, N + 1); dt= 1.0/ N; beta=.5; plt.figure(figsize=(16, 7));
np.random.seed(42); X= dt ** beta * (2 * np.random.binomial(1,.5, size=(nsim, N)) - 1)
X= np.column_stack([np.zeros(nsim), np.cumsum(X, axis=1)]); plt.subplot(1, 2, 1);
H= plt.hist(X[:, -1], bins='auto', density=True, alpha=0); plt.subplots_adjust(wspace=.3)
[plt.plot(t * dt, X[i, :], color=plt.cm.viridis(i/ nsim), lw=.5) for i in range(nsim)]
plt.plot(t * dt, np.sqrt(t * dt), 'r-', lw=3, label=''); plt.plot(t * dt, -np.sqrt(t * dt), 'r-', lw=3)
plt.plot(t * dt, np.zeros_like(t), 'k-', lw=2, label='Mean')
[plt.plot(.999, X[i, -1], 'o', ms=3, color=plt.cm.viridis(i/ nsim), alpha=.7) for i in range(nsim)]
plt.xlabel('Time'); plt.ylabel(''); plt.ylim(-2, 2); plt.xlim(0, 1); plt.gca().set_aspect('auto')
plt.subplot(1, 2, 2); be, bh= H[1], H[0]; bc= (be[:-1] + be[1:])/ 2; bw= np.diff(be)
colors= plt.cm.rainbow(np.linspace(.08,.6, len(bh)))
[plt.barh(y=bc[i], width=bh[i], height=bw[i], color=colors[i], alpha=.7) for i in range(len(bh))]
x= np.linspace(-2, 2, 100); px= (1/ np.sqrt(2 * np.pi)) * np.exp(-x**2/ 2);
plt.plot(px,x,'k-',lw=2,label='Normal density');plt.xlabel('Density');plt.ylabel('');plt.ylim(-2,2);
plt.xlim(0, max(np.max(px), np.max(bh))); plt.yticks([]); plt.tight_layout(); plt.show()
```

Listing 1.2: Python code - Brownian motion simulation.

Remark 1.3.

i) The choice of the square root in (1.2) is in fact not fortuitous. Indeed, any choice of $\pm(\Delta t)^\alpha$ with a power $\alpha \in (0, 1/2)$ would lead to explosion of the process as dt tends to zero, whereas a power $\alpha > 1/2$ would lead to a vanishing process, as can be checked by changing the value of beta in the above codes. *

ii) According to this representation, the paths of Brownian motion are not differentiable, although they are continuous by Property 2, as we have

$$\frac{\Delta B_t}{\Delta t} \simeq \frac{\pm\sqrt{\Delta t}}{\Delta t} = \pm\frac{1}{\sqrt{\Delta t}} \simeq \pm\infty, \tag{1.3}$$

see e.g. Theorem 10.3 page 153 of Schilling and Partzsch (2014).

We may now state the formal definition of Brownian motion.

Definition 1.4. *The standard Brownian motion is a stochastic process $(B_t)_{t\in\mathbb{R}_+}$ such that*

1. *$B_0 = 0$ almost surely,*
2. *The sample paths $t \mapsto B_t$ are (almost surely) continuous.*
3. *For any finite sequence of times $t_0 < t_1 < \cdots < t_n$, the increments*

$$B_{t_1} - B_{t_0}, B_{t_2} - B_{t_1}, \ldots, B_{t_n} - B_{t_{n-1}}$$

are independent.

4. *For any times $0 \leqslant s < t$, $B_t - B_s$ is normally distributed with mean zero and variance $t - s$.*

* Download the corresponding **IPython notebook**† that can be run **here** or **here**.

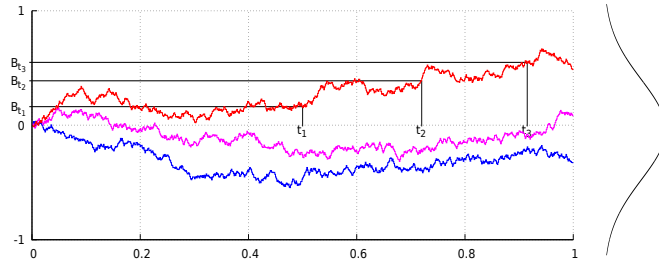


Fig. 1.7: Sample paths of a one-dimensional Brownian motion.

See e.g. Chapter 1 of Revuz and Yor (1994) and Theorem 10.28 in Folland (1999) for proofs of existence of Brownian motion as a stochastic process $(B_t)_{t\in\mathbb{R}_+}$ satisfying the Conditions *1-4* of Definition 1.4.

1.2 Market Returns

The market returns of an asset price process $(S_t)_{t\in\mathbb{R}_+}$ over time can be estimated in various ways.

Definition 1.5. *1. Standard (or arithmetic) returns are defined as*

$$\frac{\Delta S_t}{S_t} := \frac{S_{t+\Delta t} - S_t}{S_t}. \tag{1.4}$$

2. Log-returns are defined as

$$\Delta \log S_t := \log S_{t+\Delta t} - \log S_t, \quad t \geqslant 0. \tag{1.5}$$

We note that the log-returns satisfy the relation

$$\Delta \log S_t = \log \frac{S_{t+\Delta t}}{S_t} = \log\left(1 + \frac{\Delta S_t}{S_t}\right), \quad t \geqslant 0.$$

Estimating market returns

The R package quantmod can be used to fetch financial data from various sources such as Yahoo! Finance or the Federal Reserve Bank of St. Louis (FRED). It can be installed and run via the following commands.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

```r
install.packages("quantmod"); library(quantmod)
getSymbols("AAPL", src = "yahoo")
getSymbols("GOOG", src = "yahoo")
```

Listing 1.3: R code - Fetching market data using quantmod.

The yfinance package can can be used to fetch financial data from various sources such as Yahoo! Finance

```python
import pandas as pd, yfinance as yf
GOOG= yf.download("GOOG", start="2022-01-01", end="2023-12-31")
```

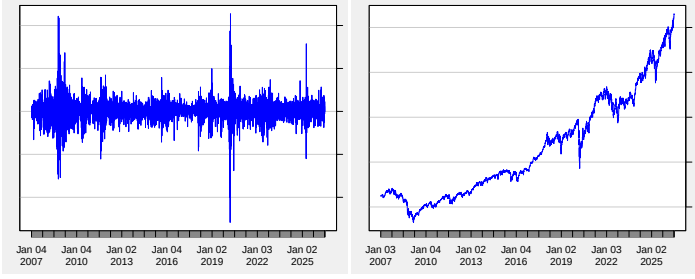
Listing 1.4: Python code - Fetching market data using yfinance.

The following script allows us to fetch market price data using the quantmod package. Market returns can be estimated using either the standard returns $\Delta S_t/S_t$ or the log-returns $\Delta \log S_t$, $t \geqslant 0$, which can be computed by the command diff(log(stock)), here with $dt = 1/365$.

```r
getSymbols("^DJI",from="2007-01-03",to=Sys.Date(),src="yahoo"); stock=Ad(`DJI`)
dev.new(width=12, height=8);
chartSeries(stock,up.col="blue",theme="white"); stock.logrtn=diff(log(stock)); # log returns
stk.rtn=(stock-lag(stock))/lag(stock);dev.new(width=12, height=8) # standard returns
chartSeries(stk.rtn,up.col="blue",theme="white"); n= length(!is.na(stk.rtn))
```

Listing 1.5: R code - Plotting market prices and returns.

The adjusted close price Ad() is the closing price computed after adjustments for applicable splits and dividend distributions.



(a) Stock returns. (b) Cumulative stock returns.

Fig. 1.8: Returns *vs.* cumulative returns.

The package yfinance can be similarly used in Python.

```python
import numpy as np, pandas as pd, yfinance as yf, matplotlib.pyplot as plt
stock_data= yf.download("^DJI", start="2007-01-03", end=pd.to_datetime("today"));
stock=stock_data["Close"];stock_logrtn=np.log(stock/stock.shift(1));
stock_rtn=stock.pct_change();fig1,ax1=plt.subplots();stock_rtn.plot(ax=ax1,c='b',lw=.3);
ax1.set_title("Returns Chart"); fig2, ax2= plt.subplots(); stock.plot(ax=ax2, c='b', lw=.5);
ax2.set_title("Stock Price Chart");plt.show();n=stock_rtn.count();
```

Listing 1.6: Python code - Plotting market prices and returns.

In what follows, we consider an increasing sequence of time instants $0 \leqslant t_0 < t_1 < \cdots < t_N$.

Proposition 1.6. *The sequence* $(S_{t_k})_{k=1,\ldots,N}$ *of market prices can be recovered from the sequence of standard returns*

$$\frac{\Delta S_{t_0}}{S_{t_0}} = \frac{S_{t_1} - S_{t_0}}{S_{t_0}}, \ldots, \frac{\Delta S_{t_{N-1}}}{S_{t_{N-1}}} = \frac{S_{t_N} - S_{t_{N-1}}}{S_{t_{N-1}}}$$

using the telescoping product identity

$$S_{t_n} = S_{t_0} \prod_{k=1}^{n} \frac{S_{t_k}}{S_{t_{k-1}}} = S_{t_0} \prod_{k=1}^{n} \left(1 + \frac{\Delta S_{t_{k-1}}}{S_{t_{k-1}}}\right), \quad n = 0, 1, \ldots, N. \tag{1.6}$$

Using log-returns, we have similarly

$$\begin{aligned} S_{t_n} &= S_{t_0} \prod_{k=1}^{n} \frac{S_{t_k}}{S_{t_{k-1}}} \\ &= S_{t_0} \prod_{k=1}^{n} e^{\log S_{t_k} - \log S_{t_{k-1}}} \\ &= S_{t_0} \prod_{k=1}^{n} e^{\Delta \log S_{t_{k-1}}}, \qquad n = 0, 1, \ldots N. \end{aligned}$$

The next R code recovers cumulative returns from standard market returns, according to (1.6).

```
stock<-stock[!is.na(stk.rtn)];stk.rtn<-stk.rtn[!is.na(stk.rtn)]
times=index(stock);dev.new(width=16,height=7);par(mfrow=c(1,2))
plot(times,stk.rtn,pch=19,xaxs="i",yaxs="i",cex=.03,col="blue", ylab="", xlab="", main=
  'Asset returns', las=1, cex.lab=1.8, cex.axis=1.8, lwd=3)
segments(x0= times, x1= times, y0= 0, y1= stk.rtn,col="blue")
plot(times,100 * cumprod(1 + as.numeric(stk.rtn)),type='l',col='black',main= "Asset
  prices",ylab="", cex=.1,cex.axis=1,las=1)
```

Listing 1.7: R code - Cumulative returns.

The next Python code recovers cumulative returns from standard market returns, according to (1.6).

```
stock= stock[~np.isnan(stock_rtn)]; stock_rtn= stock_rtn[~np.isnan(stock_rtn)]
times= stock.index; plt.subplot(1, 2, 1); plt.plot(times, stock_rtn, c='b')
plt.xlabel('Time'); plt.ylabel('Asset Returns'); plt.title('Asset Returns'); plt.xticks(rotation=90)
plt.grid(True);plt.subplot(1,2,2);cum_returns=(1+stock_rtn.astype(float)).cumprod()*100
plt.plot(times, cum_returns, c='k'); plt.xlabel('Time')
plt.ylabel('Asset Prices'); plt.title('Asset Prices'); plt.xticks(rotation=90); plt.grid(True)
plt.tight_layout(); plt.show();
```

Listing 1.8: Python code - Cumulative returns.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

1.3 Geometric Brownian Motion

The evolution of a riskless bank account value $(A_t)_{t\in\mathbb{R}_+}$ is constructed from standard returns, defined as follows:

$$\frac{\Delta A_t}{A_t} := \frac{A_{t+\Delta t} - A_t}{A_t} = r\Delta t.$$

This equation can be regarded as a discretization of the differential equation

$$A'_t = \frac{dA_t}{dt} = rA_t, \quad t \geqslant 0,$$

which has the solution

$$A_t = A_0 \mathrm{e}^{rt}, \qquad t \geqslant 0, \tag{1.7}$$

where $r > 0$ is the risk-free interest rate.*

Assumption 1.7. *Going beyond the above deterministic construction, geometric Brownian motion (GBM) was proposed as a model for stock prices by Samuelson (1965).*

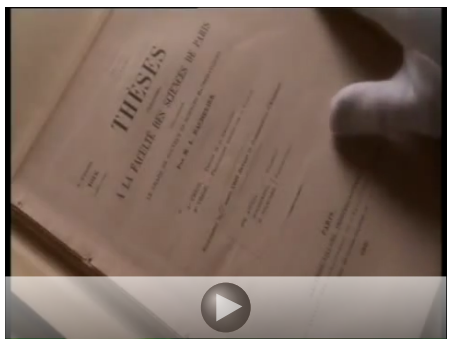


Fig. 1.9: Clark (2000) "As if a whole new world was laid out before me."†

In this framework, we will model the risky asset price process $(S_t)_{t\in\mathbb{R}_+}$ using standard returns, from the equation

* "Anyone who believes exponential growth can go on forever in a finite world is either a madman or an economist", K. E. Boulding (1973), page 248.

† Click on the figure to play the video (works in Acrobat Reader on the entire pdf file).

$$\frac{\Delta S_t}{S_t} := \frac{S_{t+\Delta t} - S_t}{S_t} = \mu \Delta t + \sigma \Delta B_t, \qquad t \geqslant 0. \tag{1.8}$$

Equation (1.8) can be solved numerically using (1.6), according to the following code.

```
N=2000; t<- 0:N; dt<- 1.0/N;mu=.5; sigma=.2; nsim<- 10; S<- matrix(0, nsim, N+1)
dB<- matrix(rnorm(nsim*N,mean=0,sd=sqrt(dt)), nsim, N+1); for (i in 1:nsim){S[i,1]=1.0;
for (j in 1:N+1){S[i,j]=S[i,j-1]*(1 + mu*dt+sigma*dB[i,j])}}
plot(t*dt, rep(0, N+1), xlab= "Time", ylab= "Geometric Brownian motion", lwd=2, ylim=
  c(min(S),max(S)), type= "l", col= 0,las=1, cex.axis=1.5,cex.lab=1.5, xaxs='i', yaxs='i')
for (i in 1:nsim){lines(t*dt, S[i, ], lwd=2, type= "l", col= i)}
```

Listing 1.9: R code - Numerical solution of (1.8).

```
import numpy as np, matplotlib.pyplot as plt;
N= 2000; t= np.arange(0, N+1); dt= 1.0/ N; mu=.5; sigma=.2; nsim= 10;
dB= np.random.normal(0, np.sqrt(dt), (nsim, N+1)); S= np.ones((nsim, N+1));
[exec("S[:, j]= S[:, j-1] * (1 + mu * dt + sigma * dB[:, j])") for j in range(1, N+1)];
plt.figure(figsize=(10, 6)); plt.plot(t * dt, np.zeros(N+1), c='k', lw=2);
[plt.plot(t * dt, S[i], lw=1, alpha=.8) for i in range(nsim)]; plt.xlabel("Time");
plt.ylabel("Geometric Brownian Motion"); plt.xlim(0, 1); plt.ylim(S.min(), S.max());
plt.xticks(fontsize=12);plt.yticks(fontsize=12);plt.grid(True,alpha=.3)
plt.title("Geometric Brownian Motion Simulations");plt.tight_layout();plt.show()
```

Listing 1.10: Python code - Numerical solution of (1.8).

The following proposition gives the explicit solution to (1.8) regarded as a discretization of the stochastic differential equation

$$\frac{dS_t}{S_t} = \frac{S_{t+dt} - S_t}{S_t} = \mu dt + \sigma dB_t.$$

Proposition 1.8 (Geometric Brownian motion)**.** *The solution of the stochastic differential equation*

$$dS_t = \mu S_t dt + \sigma S_t dB_t \tag{1.9}$$

is given by

$$S_t = S_0 \exp\left(\sigma B_t + \left(\mu - \frac{1}{2}\sigma^2\right) t\right), \qquad t \geqslant 0. \tag{1.10}$$

Proof. From (1.2), we note the following Itô multiplication table for the products of differentials dt and dB_t.

$\cdot$	dt	ΔB_t
dt	0	0
ΔB_t	0	dt

Table 1.1: Itô multiplication table.

Next, using the second order approximation

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

$$\log(1+x) \simeq x - \frac{x^2}{2}$$

as x tends to zero and the Itô Table 1.1, we have

$$\begin{aligned}
\Delta \log S_t &\simeq \log S_{t+dt} - \log S_t \\
&= \log \frac{S_{t+dt}}{S_t} \\
&= \log\left(1 + \frac{S_{t+dt} - S_t}{S_t}\right) \\
&= \log\left(1 + \frac{\Delta S_t}{S_t}\right) \\
&= \log(1 + \mu dt + \sigma \Delta B_t) \\
&= \mu dt + \sigma \Delta B_t - \frac{1}{2}(\mu dt + \sigma \Delta B_t)^2 \\
&\simeq \mu dt + \sigma \Delta B_t - \frac{\mu^2}{2}(dt)^2 - \mu\sigma \Delta B_t \cdot dt - \frac{\sigma^2}{2}(\Delta B_t)^2 \\
&\simeq \mu dt - \frac{\sigma^2}{2} dt + \sigma \Delta B_t, \qquad t \geqslant 0.
\end{aligned}$$

By integration over the time interval $[0, t]$ we find

$$\begin{aligned}
\log S_t &= \log S_0 + \int_0^t d \log S_s \\
&= \int_0^t \mu ds - \frac{\sigma^2}{2} \int_0^t ds + \sigma \int_0^t \Delta B_s \\
&= \mu t - \frac{\sigma^2 t}{2} + \sigma B_t,
\end{aligned}$$

which yields (1.10) after exponentiation. □

Figure 1.10 presents an illustration of the geometric Brownian process of Proposition 1.8 according to the following codes.

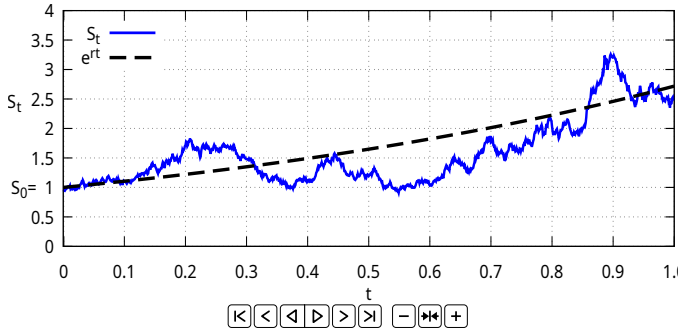


Fig. 1.10: Geometric Brownian motion started at $S_0 = 1$, with $\mu = r = 1$ and $\sigma^2 = 0.5$.*

```
N=1000; t<- 0:N; dt<- 1.0/N; sigma=.2; mu=.5; dB<- rnorm(N,mean=0,sd=sqrt(dt));
plot(t*dt, exp(mu*t*dt), xlab= "time", ylab= "Geometric Brownian motion", type= "l", ylim=
  c(.75,2), col= 1,lwd=3)
lines(t*dt, exp(sigma*c(0,cumsum(dB))+mu*t*dt-sigma*sigma*t*dt/2),xlab= "time",type=
  "l",ylim=c(0, 4), col= 'blue', lwd=1.2, xaxs='i', yaxs='i')
```

Listing 1.11: R code - Geometric Brownian motion simulation.

```
import numpy as np, matplotlib.pyplot as plt
N= 1000; t= np.arange(0, N+1); dt= 1.0/ N; sigma=.2; mu=.5
dB= np.random.normal(0, np.sqrt(dt), N)
GBM= np.exp(sigma* np.concatenate(([0],np.cumsum(dB)))+ mu* t* dt- (sigma**2)* t* dt/ 2)
plt.plot(t*dt, np.exp(mu * t * dt), c='b', lw=3, label='Expectation')
plt.plot(t*dt, GBM, c='r', lw=1.5, label=''); plt.xlabel(''); plt.ylabel('')
plt.xlim(0, N*dt); plt.ylim(.75,2); plt.legend(); plt.grid(True); plt.show()
```

Listing 1.12: Python code - Geometric Brownian motion simulation.

Multiple sample paths of the solution (1.10) to (1.9) can also be generated using the following code.

```
N=2000; t<- 0:N; dt<- 1.0/N; mu=.5;sigma=.2; nsim<- 10; par(oma=c(0,1,0,0))
dB<- matrix(rnorm(nsim*N,mean=0,sd=sqrt(dt)), nsim, N)
dB<- cbind(rep(0, nsim), t(apply(dB, 1, cumsum)))
for (i in 1:nsim){dB[i,]<- exp(mu*t*dt+sigma*dB[i,]-sigma*sigma*t*dt/2)}
plot(t*dt, exp(mu*t*dt), xlab= "Time", ylab= "Geometric Brownian motion", lwd=3, ylim=
  c(min(dB),max(dB)), type= "l", col= 1,las=1,cex.axis=1.5,cex.lab=1.6, xaxs='i', yaxs='i')
for (i in 1:nsim){lines(t*dt, dB[i, ], lwd=1.2, type= "l", col= 2+i%%7)}
```

Listing 1.13: R code - Multiple paths generation.

* The animation works in Acrobat Reader on the entire pdf file.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

```
import numpy as np, matplotlib.pyplot as plt;
N= 2000; t= np.arange(0, N+1); dt= 1.0/ N; mu=.5; sigma=.2; nsim= 10;
dB= np.random.normal(0, np.sqrt(dt), (nsim, N));
dB= np.concatenate((np.zeros((nsim, 1)), np.cumsum(dB, axis=1)), axis=1);
[exec("dB[i, :]=np.exp(mu*t*dt+sigma*dB[i, :]-(sigma**2)*t*dt/2)") for i in range(nsim)];
plt.plot(t*dt,np.zeros(N+1),c='k',lw=1); plt.xlabel('Time'); plt.ylabel('');
plt.xlim(0, N*dt); plt.ylim(np.min(dB), np.max(dB)); plt.xticks(fontsize=12);
plt.yticks(fontsize=12); [plt.plot(t*dt, dB[i, :], lw=1) for i in range(nsim)]; plt.show()
```

Listing 1.14: Python code - Multiple paths generation.

Figure 1.11 presents sample paths of geometric Brownian motion.

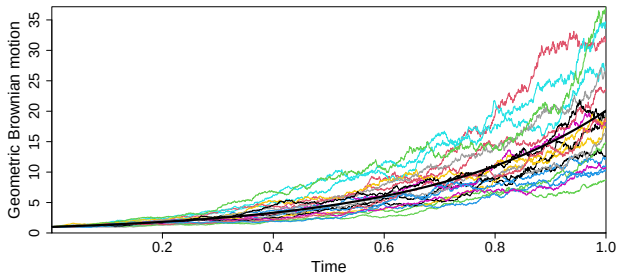


Fig. 1.11: Twenty sample paths of geometric Brownian motion $(S_t)_{t\in\mathbb{R}_+}$.

In Proposition 1.9, we compute the probability distribution of geometric Brownian motion at any given time.

Proposition 1.9. *At any time $T > 0$, the random variable*

$$S_T := S_0 \mathrm{e}^{\sigma B_T + (\mu - \sigma^2/2)T}$$

has the lognormal distribution *with probability density function*

$$x \longmapsto f(x) = \frac{1}{x\sigma\sqrt{2\pi T}} \mathrm{e}^{-(-(\mu-\sigma^2/2)T + \log(x/S_0))^2/(2\sigma^2 T)}, \quad x > 0, \tag{1.11}$$

with log-variance σ^2 and log-mean $(\mu - \sigma^2/2)T + \log S_0$, see Figure 1.13.

Proof. For all $x \in \mathbb{R}$, we have

$$\begin{aligned} \mathbb{P}(S_T \leqslant x) &= \mathbb{P}\big(S_0 \mathrm{e}^{\sigma B_T + (\mu - \sigma^2/2)T} \leqslant x\big) \\ &= \mathbb{P}\left(\sigma B_T + \left(\mu - \frac{\sigma^2}{2}\right) T \leqslant \log \frac{x}{S_0}\right) \end{aligned}$$

$$
\begin{aligned}
&= \mathbb{P}\left(B_T \leqslant \frac{1}{\sigma}\left(\log\frac{x}{S_0} - \left(\mu - \frac{\sigma^2}{2}\right) T \right) \right) \\
&= \int_{-\infty}^{(\log(x/S_0)-(\mu-\sigma^2/2)T)/\sigma} \mathrm{e}^{-y^2/(2T)} \frac{dy}{\sqrt{2\pi T}} \\
&= \int_{-\infty}^{(\log(x/S_0)-(\mu-\sigma^2/2)T)/(\sigma\sqrt{T})} \mathrm{e}^{-z^2/2} \frac{dz}{\sqrt{2\pi}} \\
&= \Phi\left(\frac{1}{\sigma\sqrt{T}} \left(\log\frac{x}{S_0} - \left(\mu - \frac{\sigma^2}{2}\right) T \right) \right),
\end{aligned}
$$

where

$$
\Phi(x) := \int_{-\infty}^{x} \mathrm{e}^{-y^2/2} \frac{dy}{\sqrt{2\pi}}, \qquad x \in \mathbb{R}, \tag{1.12}
$$

denotes the standard Gaussian Cumulative Distribution Function (CDF) of a standard normal random variable $X \simeq \mathcal{N}(0,1)$, with the relation

$$
\Phi(-x) = 1 - \Phi(x), \qquad x \in \mathbb{R}.
$$

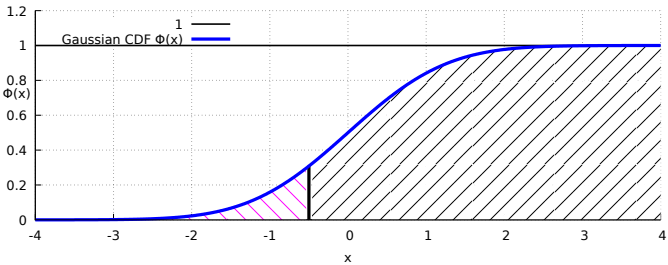


Fig. 1.12: Graph of the Gaussian Cumulative Distribution Function (CDF).

After differentiation with respect to x, we find the lognormal probability density function

$$
\begin{aligned}
f(x) &= \frac{\mathrm{d}\mathbb{P}(S_T \leqslant x)}{dx} \\
&= \frac{\partial}{\partial x} \int_{-\infty}^{(\log(x/S_0)-(\mu-\sigma^2/2)T)/\sigma} \mathrm{e}^{-y^2/(2T)} \frac{dy}{\sqrt{2\pi T}} \\
&= \frac{\partial}{\partial x} \Phi\left(\frac{1}{\sigma\sqrt{T}} \left(\log\frac{x}{S_0} - \left(\mu - \frac{\sigma^2}{2}\right) T \right) \right) \\
&= \frac{1}{x\sigma\sqrt{T}} \varphi\left(\frac{1}{\sigma\sqrt{T}} \left(\log\frac{x}{S_0} - \left(\mu - \frac{\sigma^2}{2}\right) T \right) \right)
\end{aligned}
$$

$$= \frac{1}{x\sigma\sqrt{2\pi T}} \mathrm{e}^{-(-(\mu-\sigma^2/2)T+\log(x/S_0))^2/(2\sigma^2 T)}, \qquad x > 0,$$

where

$$\varphi(y) = \Phi'(y) := \frac{1}{\sqrt{2\pi}} \mathrm{e}^{-y^2/2}, \qquad y \in \mathbb{R},$$

denotes the standard Gaussian probability density function. □

The mean and variance of S_t can be computed as

$$\mathbb{E}[S_t] = S_0 \mathrm{e}^{\mu t} \quad \text{and} \quad \mathrm{Var}[S_t] = S_0^2 \mathrm{e}^{2\mu t}\big(\mathrm{e}^{\sigma^2 t} - 1\big), \quad t \geqslant 0. \tag{1.13}$$

The next code is generating sample paths of geometric Brownian motion together with the histogram of terminal values fitted to the lognormal probability density function (1.11), see Figure 1.13.

```
N=1000; t<- 0:N; dt<- 1.0/N; ns<- 100 # using Bernoulli samples
sigma=.2;r=.5;a=(1+r*dt)*(1-sigma*sqrt(dt))-1;b=(1+r*dt)*(1+sigma*sqrt(dt))-1
X<- matrix(a+(b-a)*rbinom( ns * N, 1,.5), ns, N); dev.new(width=16,height=7);
X<-cbind(rep(0,ns),t(apply((1+X),1,cumprod))); X[,1]=1;H<-hist(X[,N],plot=FALSE);
layout(matrix(c(1,2), nrow=1, byrow= TRUE)); par(mar=c(2,2,2,0), oma= c(2, 2, 2, 2))
plot(t*dt,X[1,],xlab="time",ylab="",type="l",ylim=c(.8,3),col= 0,xaxs='i',las=1,cex.axis=1.6)
for (i in 1:ns){lines(t*dt,X[i,],xlab="time",type="l",col=i)};pl=rainbow(20,start=.08,end=.6)
lines((1+r*dt)^t, type="l", lty=1, lwd=3,xlab="",ylab="", main="")
for (i in 1:ns){points(.999, X[i,N], pch=1, lwd= 5, col= i)}; x<- seq(.01,3, length=100);
px<- exp(-(-(r-sigma^2/2)+log(x))^2/2/sigma^2)/x/sigma/sqrt(2*pi); par(mar= c(2,2,2,2))
plot(NULL , xlab="", ylab="", xlim= c(0, max(px,H$density)),ylim=c(.8,3),axes=F, las=1)
rect(0, H$breaks[1:(length(H$breaks) - 1)], col=pl, H$density, H$breaks[2:length(H$breaks)])
lines(px,x, type="l", lty=1, lwd=3,xlab="",ylab="", main="")
```

Listing 1.15: R code - GBM simulation with lognormal histogram.

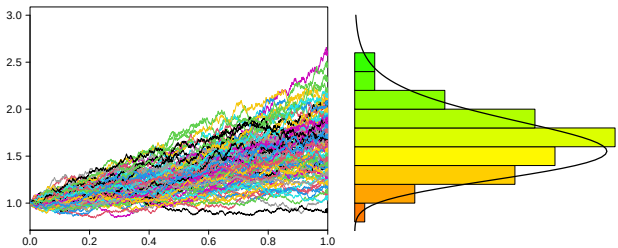


Fig. 1.13: Statistics of geometric Brownian paths *vs.* lognormal distribution.

```
import numpy as np, matplotlib.pyplot as plt; N, ns, s, r= 1000, 100, .2, .5; dt= 1/N;
t= np.arange(N+1); a, b= (1+r*dt)*(1+np.array([-1,1])*s*np.sqrt(dt)) - 1
X=a+(b-a)*np.random.binomial(1,.5,(ns,N));X=np.c_[np.ones(ns),np.cumprod(1+X,1)]
plt.figure(figsize=(16,7)); H= plt.hist(X[:,-1], bins='auto', density=True, alpha=0)
plt.subplot(1,2,1); plt.subplots_adjust(wspace=.3); c= plt.cm.viridis(np.arange(ns)/ns)
[plt.plot(t*dt, X[i], color=c[i]) for i in range(ns)]; plt.plot(t*dt, (1+r*dt)**t, 'k', lw=3)
[plt.plot(.999, X[i,-1], 'o', ms=3, color=c[i], alpha=.7) for i in range(ns)]; plt.xlim(0,1);
plt.ylim(.8,3); plt.xlabel('time'); plt.subplot(1,2,2); be, bh= H[1], H[0];
bc, bw=(be[:-1]+be[1:])/2,np.diff(be);cl=plt.cm.rainbow(np.linspace(.08,.6,len(bh)))
[plt.barh(bc[i],bh[i],bw[i],color=cl[i],alpha=.7) for i in range(len(bh))];x=np.linspace(.01,3,100);
px=np.exp(-(np.log(x)-(r-s*s/2))**2/(2*s*s))/(x*s*np.sqrt(2*np.pi));plt.plot(px,x,'k',lw=3);
plt.xlim(0,max(px.max(),bh.max()));plt.ylim(.8,3);plt.yticks([]);plt.tight_layout();plt.show()
```

Listing 1.16: Python code - GBM simulation with lognormal histogram.

Time-dependent coefficients

The above construction can be extended to time-dependent interest rate, drift and volatility processes $(r(t))_{t\in\mathbb{R}_+}$, $(\mu(t))_{t\in\mathbb{R}_+}$ and $(\sigma(t))_{t\in\mathbb{R}_+}$. In this case, we let $(A_t)_{t\in\mathbb{R}_+}$ be the risk-free asset with price given by

$$\frac{dA_t}{A_t} = r(t)dt, \qquad A_0 = 1, \qquad t \geqslant 0,$$

i.e.

$$A_t = A_0 \mathrm{e}^{\int_0^t r(s)ds}, \qquad t \geqslant 0,$$

and the price process $(S_t)_{t\in[0,T]}$ is defined by the stochastic differential equation

$$dS_t = \mu(t)S_t dt + \sigma(t)S_t dB_t, \qquad t \geqslant 0,$$

i.e., in integral form,

$$S_t = S_0 + \int_0^t \mu(u)S_u du + \int_0^t \sigma(u)S_u dB_u, \qquad t \geqslant 0,$$

with solution

$$S_t = S_0 \exp\left(\int_0^t \sigma(s)dB_s + \int_0^t \left(\mu(s) - \frac{1}{2}\sigma^2(s)\right) ds\right), \quad t \geqslant 0,$$

where $\int_0^t \sigma(s)dB_s$ denotes the stochastic integral of $(\sigma(s))_{s\in[0,t]}$ with respect to Brownian motion. In this case, the mean and variance expressions in (1.13) can be extended as

$$\mathbb{E}[S_t] = S_0 \exp\left(\int_0^t \mu(s)ds\right)$$

and

$$\mathrm{Var}[S_t] = S_0^2 \exp\left(2\int_0^t \mu(s)ds\right)\left(\exp\left(\int_0^t \sigma^2(s)ds\right) - 1\right).$$



In addition, median and mode of S_t are respectively given by

$$S_0 \exp\left(\int_0^t \left(\mu(s) - \frac{1}{2}\sigma^2(s)\right) ds\right) \quad \text{and} \quad S_0 \exp\left(\int_0^t \left(\mu(s) - \frac{3}{2}\sigma^2(s)\right) ds\right),$$

$t \geqslant 0$.

1.4 Distribution of Market Returns

Geometric Brownian motion *vs.* market data

In this section, we assess the properties of Brownian and geometric Brownian motions We start with a comparison of geometric Brownian motion simulations to asset price data using the the following code.

```
library(quantmod);getSymbols("0005.HK",from="2016-02-15",to="2017-05-11",src="yahoo")
MP=Ad(`0005.HK`);r=(MP-lag(MP))/lag(MP);mu=mean(r[-1]);s=sd(r[-1]);N=length(MP);
X=matrix(mu+s*rnorm(N-1)/2,1);X=as.numeric(MP[1])*cbind(0,t(apply(1+X,1, cumprod)));
X[,1]=MP[1];GBM=xts(X[1,],index(MP));pars=chart_pars();pars$cex=1.4;th=chart_theme();
th$col$line.col="blue"; th$rylab=FALSE; chart_Series(MP, pars=pars, theme=th);
ddexp=xts(as.numeric(MP[1])*exp(mu*seq_len(N)),index(MP));dev.new(width=16,height=8);
par(mfrow=c(1,2)); add_TA(exp(log(ddexp)), on=1, lwd=4, legend=NULL)
g=chart_Series(GBM,pars=pars,theme=th);yl=g$get_ylim();g$set_ylim(yl);
g=add_TA(exp(log(ddexp)),on=1,lwd=4);yl[[2]]=structure(range(MP),fixed=TRUE);g
```

Listing 1.17: R code - Geometric Brownian motion *vs.* market data.

```
import numpy as np, pandas as pd, yfinance as yf; from matplotlib import pyplot as plt
data= yf.download("0005.HK", start="2016-02-15", end="2017-05-11", progress=False)
MP= data['Close']; returns= (MP - MP.shift(1))/ MP
sigma= np.std(returns[1:])[0]; mu= np.mean(returns[1:]); N= len(MP);
t= np.arange(0, N); X= np.array(mu + sigma * np.random.normal(0, 1, N-1)/ 2)
X= np.concatenate(([MP.iloc[0].item()], MP.iloc[0].item()*np.cumprod(1 + X)))
x= np.arange(100, 100 + N); dates= MP.index; GBM= pd.Series(X, index=dates)
fig= plt.figure(figsize=(16, 8)); ax1= fig.add_subplot(121)
ax1.plot(MP, c='b', lw=2); plt.grid(True); ax1.set_ylabel('Price'); ax1.set_xlabel('Date');
ylim1= ax1.get_ylim(); ax2= fig.add_subplot(122); ax2.plot(GBM, c='b', lw=2);
ax2.set(xlabel='',ylabel='');ax2.set_ylim(ylim1);plt.tight_layout();plt.grid(True); plt.show()
```

Listing 1.18: Python code - Geometric Brownian motion *vs.* market data.

Figure 1.14 presents a graph of the HSBC market price, which is compared to the geometric Brownian motion simulation of Figure 1.11 in Figure 1.15.

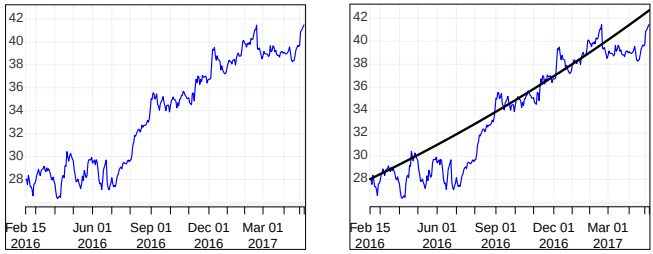


Fig. 1.14: HSBC Market price.

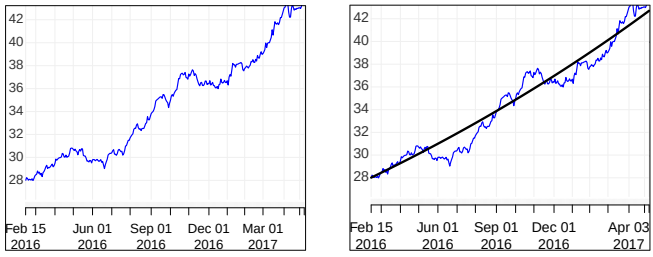


Fig. 1.15: Graph of simulated geometric Brownian motion.

The coefficients of a geometric Brownian motion can be estimated from observed market data.

```
install.packages("Sim.DiffProc");library("Sim.DiffProc");
fx<- expression( theta[1]*x ); gx<- expression( theta[2]*x )
fitsde(data=as.ts(MP),drift=fx,diffusion=gx,start=list(theta1=.01, theta2=.01), pmle="euler")
```

Listing 1.19: code - GBM Parameter estimation.

```
import numpy as np; from scipy.optimize import minimize; from scipy.stats import norm;
logl= lambda params, dat, dt: (lambda theta1, theta2: (1e10 if np.any((theta2 * dat[:-1]) <= 0)
  else -np.sum(norm.logpdf(dat[1:], loc=dat[:-1] + (theta1 * dat[:-1]) * dt, scale=(theta2 *
  dat[:-1]) * np.sqrt(dt))))(*params)
dat=MP.values; dt=1/252; init_p=[.01,.01];
rs=minimize(logl,init_p,(dat,dt),'L-BFGS-B',bounds=[(None,None),(1e-10,None)])
theta1_est,theta2_est=rs.x; print(f"Estimated theta1 (drift): {theta1_est:.6f}");
print(f"Estimated theta2 (diffusion): {theta2_est:.6f}")
```

Listing 1.20: Python code - GBM Parameter estimation.

Market returns *vs.* Brownian increments

By inverting Relation (1.8), we can recover the Brownian increment ΔB_t from the market returns ΔS_t, as

$$\Delta B_t = \frac{1}{\sigma}\left(\frac{\Delta S_t}{S_t} - \mu\Delta t\right), \qquad t \geqslant 0,$$

which, by integration, yields

$$B_{t_n} = B_0 + \sum_{k=1}^{n} \Delta B_{t_k} = \frac{1}{\sigma}\sum_{k=1}^{n}\left(\frac{\Delta S_{t_k}}{S_{t_k}} - \mu\Delta t\right). \tag{1.14}$$

The following code * implements (1.14) using 72 yearly return graphs of the S&P 500 index from 1950 to 2022, see Figure 1.16, which can be compared to the Brownian simulation of Figure 1.6. The histogram of year-end cumulative returns is then compared to a Gaussian probability density function calibrated using mean and variance.

```
library(quantmod);getSymbols("^GSPC",from="1950-01-01",to="2022-12-31",src="yahoo")
stock=Cl(`GSPC`);s=0;y=0;j=0;ct=0;N=240;nsim=72;X=matrix(0,nsim,N)
for (i in 1:nrow(GSPC)){if (s==0 && grepl('-01-0',index(stock[i]))) {if (ct==0 || X[y,N]>0)
{y=y+1;j=1;s=1;ct=ct+1;}};if (j<=N) {X[y,j]=as.numeric(stock[i]);}
if (grepl('-02-0',index(stock[i]))) {s=0;};j=j+1};t=0:(N-1);dt=1/N;dev.new(width=16,height=7);
layout(t(1:2));par(mar=c(2,2,2,0),oma=c(2,2,2,2));m=mean(X[,N]/X[,1]-1);sg=sd(X[,N]/X[,1]-1)
plot(t*dt,X[1,]/X[1,1]-1-m*t*dt,xlab='',ylab='',type='l',ylim=c(-.5,.5),col=0,xaxs='i',las=1,
  cex.axis=1.6);for (i in 1:nsim){lines(t*dt,X[i,]/X[i,1]-1-m*t*dt,type='l',col=i)}
lines(t*dt,sg*sqrt(t*dt),lty=1,col="red",lwd=3);lines(t*dt,-sg*sqrt(t*dt),lty=1,col="red",lwd=3)
lines(t*dt,0*t,lty=1,col='black',lwd=2);x=seq(-.5,.5,l=100);px=dnorm(x,0,sg);
for (i in 1:nsim){points(.999,X[i,N]/X[i,1]-1-m*N*dt,pch=1,lwd=5,col=i)}
par(mar=c(2,2,2,2)); H=hist(X[,N]/X[,1]-1-m*N*dt,plot=F);
plot(NULL,xlab='',ylab='',xlim=c(0,max(px,H$density)),ylim=c(-.5,.5),axes=F)
rect(0,H$breaks[-length(H$breaks)],col=rainbow(20,start=.08,end=.6),H$density,H$breaks[-1])
lines(px,x,lty=1,col='black',lwd=2)
```

Listing 1.21: R code - Historical returns *vs.* normal distribution.

```
import yfinance as yf,numpy as np,matplotlib.pyplot as plt;from scipy.stats import norm, skew
gspc=yf.download("^GSPC",start="1950-01-01",end="2022-12-31")
stk=gspc['Close'];s,y,j,ct=0,0,0,0;N,ns=240,72;X=np.zeros((ns,N));B=np.zeros((ns,N))
__=[(dat__s:=str(stk.index[i])[:10],s==0 and '-01-0' in dat__s and (ct==0 or X[y-1,N-1]>0) and
  (y:=y+1,j:=0,s:=1,ct:=ct+1),j<N and y>0 and y<=ns and
  (X.___setitem___((y-1,j),stk.iloc[i])),'-02-0' in dat__s and (s:=0),j:=j+1) for i in range(len(stk))]
t=np.arange(N);dt=1./N;fig,(ax1,ax2)=plt.subplots(1,2,figsize=(16,7))
__ =[B.___setitem___((i,slice(1,None)),np.cumsum((X[i,1:]-X[i,:-1])/X[i,:-1])) for i in range(ns)]
mu=np.mean(B[:,-1])*dt;sigma=np.std(B[:,-1])*dt**.5
__ =[B.___setitem___((i,slice(None)),(B[i,:]-mu*t)*dt**.5/sigma) for i in range(ns)]
__ =[ax1.plot(t*dt,B[i,:],color=plt.cm.viridis(i/ns)) or ax1.scatter(.999,B[i,-1],s=50,
color=plt.cm.viridis(i/ns)) for i in range(ns)];ax1.plot(t*dt,(t*dt)**.5,'r-',lw=3);
ax1.plot(t*dt,-(t*dt)**.5,'r-',lw=3);ax1.axhline(0,color='black',lw=2);f__v=B[:,-1];
ax1.set__ylim(-3,3);ax1.set__xlim(0,1);x=np.linspace(-3,3,99);px=norm.pdf(x);
cts,bins=np.histogram(f__v,bins=20,range=(-3,3));bc=(bins[:-1]+bins[1:])/2;
dens=cts/(len(f__v)*(bins[1]-bins[0]));ax2.plot(px,x,'k-',lw=2);ax2.set__ylim(-3,3)
__ =[ax2.barh(bc[i],dens[i],height=(bins[1]-bins[0]),color=plt.cm.rainbow(i/20),
left=0,alpha=.7) for i in range(len(bins)-1)];ax2.set__xlim(0,max(px.max(),dens.max()));
plt.tight__layout(pad=0);plt.show();print(f"Skewness: {skew(f__v):.6f}")
```

Listing 1.22: Python **3.8** code - Historical returns *vs.* normal distribution.

* Download the corresponding **IPython notebook**† that can be run **here** or **here**.

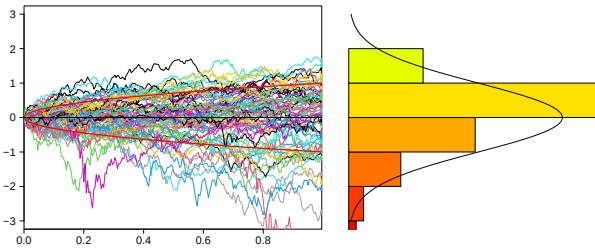


Fig. 1.16: Statistics of 72 yearly return graphs of the S&P 500 from 1950 to 2022.

Market returns *vs.* Gaussian and power tails

Consider for example the market returns data obtained from fetching DJI and STI index data using the following script.

```
library(quantmod); dev.new(width=16,height=8)
getSymbols("^STI",from="1990-01-03",to="2015-01-03",src="yahoo");stock=Ad(`STI`);
getSymbols("^DJI",from="1990-01-03",to=Sys.Date(),src="yahoo");stock=Ad(`DJI`);
stk.rtn=diff(log(stock));rt<-as.vector(stk.rtn);m=mean(rt,na.rm=TRUE);s=sd(rt,na.rm=TRUE);
ts=index(stk.rtn);n=sum(is.na(rt))+sum(!is.na(rt));x=seq(1,n);y=rnorm(n,mean=m,sd=s)
plot(ts,rt,pch=19,xaxs="i",yaxs="i",cex=.03,col="blue", ylab="", xlab="", main='', las=1,
  cex.lab=1.8, cex.axis=1.8, lwd=3)
segments(x0=ts,x1=ts,y0=0,y1=rt,col="blue");points(ts,y,pch=19,cex=.3,col="red")
abline(h=m+3*s, lwd=1);abline(h=m, lwd=1);length(rt[abs(rt-m)>3*s])/length(stk.rtn);
length(y[abs(y-m)>3*s])/length(y);2*(1-pnorm(3*s,0,s)); abline(h=m-3*s, lwd=1)
```

Listing 1.23: R code - Market returns *vs.* Gaussian samples.

```
import numpy as np, pandas as pd, yfinance as yf, matplotlib.pyplot as plt;
from scipy.stats import norm
sti= yf.download("^STI", start="1990-01-03", end="2015-01-03", progress=False)['Close']
dji= yf.download("^DJI", start="1990-01-03", progress=False)['Close']
stock= dji; stock_rtn= np.log(stock).diff(); returns= stock_rtn.dropna()
m,s=returns.mean().item(),returns.std().item();times=returns.index;n=len(returns);
y=np.random.normal(m,s,n);plt.figure(figsize=(16,8));plt.plot(times,returns,'bo',ms=1,alpha=.5)
plt.vlines(times,0,returns,colors='b',lw=.5); plt.plot(times,y,'ro',ms=2,alpha=.5)
plt.axhline(m+3*s,c='k',lw=1);plt.axhline(m,c='k',lw=1); plt.axhline(m-3*s,c='k',lw=1)
plt.ylim(m - 4*s, m + 4*s); plt.xlabel(''); plt.ylabel(''); plt.tight_layout(); plt.show()
p_emp= len(returns[np.abs(returns - m) > 3*s])/ n
p_norm= len(y[np.abs(y - m) > 3*s])/ n; p_theor= 2 * (1 - norm.cdf(3*s, 0, s))
print(f"Empirical: {p_emp:.6f}, Normal sample: {p_norm:.6f}, Theoretical:{p_theor:.6f}")
```

Listing 1.24: Python code - Market returns *vs.* Gaussian samples.

Figure 1.17 shows the mismatch between the distributional properties of market log-returns *vs.* standardized Gaussian returns, which tend to underestimate the probabilities of extreme events. Note that when $X \simeq \mathcal{N}(0, \sigma^2)$,

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

99.73% of samples of X are falling within the interval $[-3\sigma, +3\sigma]$, *i.e.* $\mathbb{P}(|X| \leqslant 3\sigma) = 0.9973002$.

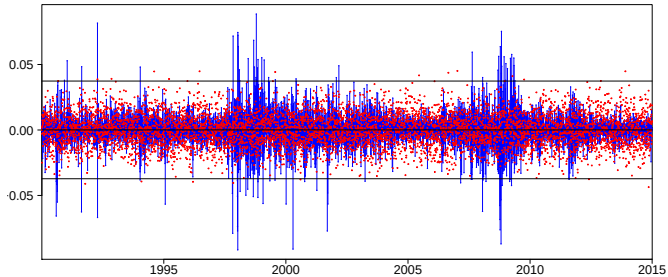


Fig. 1.17: Market returns (blue) *vs.* normalized Gaussian returns (red).

```
stock.ecdf=ecdf(as.vector(stk.rtn)); x<- seq(-.15,.15, length=200); px<- pnorm((x-m)/s)
dev.new(width=16,height=8);plot(stock.ecdf, xlab= '', col="red",ylab= '', ylim=c(-.002,1.002),
  main= '', xaxs="i", yaxs="i", las=1, cex.lab=1.8, cex.axis=1.8, lwd=4)
lines(x, px, type="l", lty=2, col="blue",xlab="",ylab="", main="", lwd=4); grid(lwd= 3)
legend("topleft",c("Emp. CDF","Gaussian CDF"),col=c("red","blue"),lty=1:2,cex=2,lwd=4)
```

Listing 1.25: code - Empirical CDF *vs.* normal CDF.

```
import numpy as np, matplotlib.pyplot as plt; from scipy.stats import norm
data=stock_rtn.dropna().values; n=len(data); data_sorted=np.sort(data)
uniq,cts=np.unique(data_sorted,return_counts=True); y=np.cumsum(cts)/n
x_ecdf=np.concatenate(([uniq[0]],uniq)); y_ecdf=np.concatenate(([.0],y));
x=np.linspace(-.15,.15,200);px=norm.cdf((x-m)/s);plt.figure(figsize=(16,8));plt.tight_layout()
plt.step(x_ecdf,y_ecdf,where='post',c='r',lw=4,label='Empirical CDF');plt.grid(True,lw=3)
plt.plot(x,px,'b--',lw=4,label='Gaussian CDF');plt.ylim(-.002,1.002);plt.xlim(-.15,.15)
plt.legend(loc='upper left',fontsize=18);plt.tick_params(axis='both',labelsize=18);plt.show()
```

Listing 1.26: Python code - Empirical CDF *vs.* normal CDF.

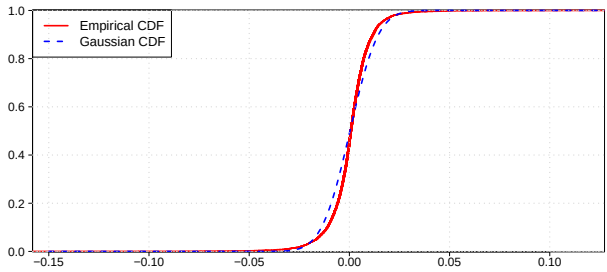


Fig. 1.18: Empirical *vs.* Gaussian CDF.

The following Quantile-Quantile graph is plotting the normalized empirical quantiles against the standard Gaussian quantiles, and is obtained with the `qqnorm(returns)` command.

```
dev.new(width=16,height=8)
qqnorm(stk.rtn,col='blue',xaxs="i",yaxs="i",las=1,cex.lab=1.4,cex.axis=1,pch=16);grid(lwd=2)
```

Listing 1.27: R code - Quantile-quantile plot.

```
import matplotlib.pyplot as plt, numpy as np; from scipy import stats
plt.figure(figsize=(16, 8)); returns_values = returns.values.flatten()
stats.probplot(returns_values, dist="norm", plot=plt); qq_plot = plt.gca()
plt.grid(True, lw=2); plt.gca().set_xlabel('Theoretical Quantiles', fontsize=14)
plt.gca().set_ylabel('Sample Quantiles', fontsize=14)
plt.gca().tick_params(axis='both',which='major',labelsize=11);plt.tight_layout();plt.show()
```

Listing 1.28: Python code - Quantile-quantile plot.

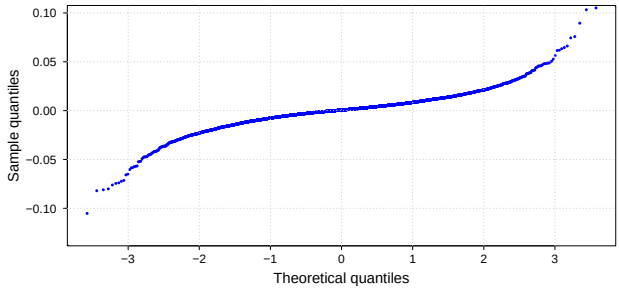


Fig. 1.19: Quantile-Quantile plot.

The Kolmogorov–Smirnov test is used to test the normality of market returns. The following application of the test clearly rejects the null (normality) hypothesis of market returns, and does not reject that hypothesis for the simulated Gaussian returns.

```
n= sum(is.na(stk.rtn))+sum(!is.na(stk.rtn));x=seq(1,n);y=rnorm(n,mean=m,sd=s)
ks.test(y,"pnorm",mean=m,sd=s); ks.test(stk.rtn,"pnorm",mean=m,sd=s)
```

Listing 1.29: R code - Kolmogorov–Smirnov test.

```
One-sample Kolmogorov--Smirnov test

data:  returns
D = 0.075577, p-value < 2.2e-16
alternative hypothesis: two-sided
```

```python
import numpy as np, pandas as pd; from scipy import stats
returns_clean= returns_values[~np.isnan(returns_values)]; n= len(returns_clean);
x= np.arange(1, n + 1); m= returns_clean.mean(); s= returns_clean.std()
y=np.random.normal(loc=m,scale=s,size=n);ks_test_y=stats.kstest(y,'norm',args=(m,s))
ks_test_returns=stats.kstest(returns_clean,'norm',args=(m,s))
print(f"Statistic: {ks_test_y.statistic:.6f}, p-value: {ks_test_y.pvalue:.6f}")
print(f"Statistic: {ks_test_returns.statistic:.6f}, p-value: {ks_test_returns.pvalue:.6f}")
```

Listing 1.30: Python code - Kolmogorov–Smirnov test.

The mismatch in distributions observed in Figures 1.17-1.19 can be further illustrated by the empirical probability density plot in Figure 1.20, which is obtained from the following code.

```r
dev.new(width=16,height=8); x<- seq(-.25,.25, length=1000); qx<- dnorm(x,mean=m,sd=s)
stk.dens=density(stk.rtn,na.rm=TRUE)
plot(stk.dens, xlab= 'x', lwd=4, col="red",ylab= '', main= '', xlim=c(-.1,.1), ylim=c(0,65),
  xaxs="i", yaxs="i", las=1, cex.lab=1.8, cex.axis=1.8)
lines(x, qx, type="l", lty=2, lwd=4, col="blue",xlab='',ylab='',main=''); grid(lwd=2)
legend("topleft",c("Emp. density","Gaussian density"),col=c("red","blue"),lty=1:2,cex=1.2)
```

Listing 1.31: R code - Empirical *vs.* normalized Gaussian density.

```python
import numpy as np, matplotlib.pyplot as plt; from scipy.stats import gaussian_kde, norm
data=stock_rtn.dropna().values.flatten(); m,s=data.mean(),data.std()
dens=gaussian_kde(data); x=np.linspace(-.25,.25,1000); y_emp=dens(x)
y_gauss=norm.pdf(x,loc=m,scale=s)
plt.figure(figsize=(16,8)); plt.plot(x,y_emp,'r',lw=4,label='Empirical density')
plt.plot(x,y_gauss,'b--',lw=4,label='Gaussian density'); plt.xlim(-.1,.1); plt.ylim(0,65)
plt.xlabel('x',fontsize=18); plt.ylabel('Density',fontsize=18); plt.legend(fontsize=16)
plt.grid(True,lw=2); plt.tick_params(axis='both',labelsize=16); plt.tight_layout(); plt.show()
```

Listing 1.32: Python code - Empirical *vs.* normalized Gaussian density.

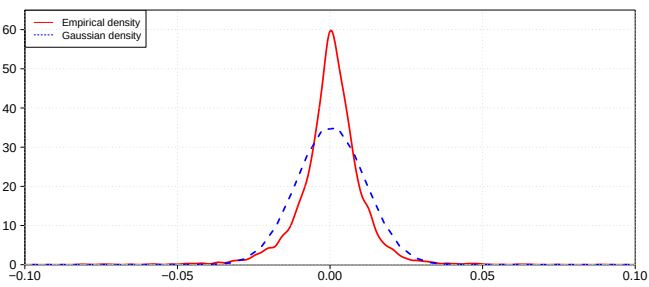


Fig. 1.20: Empirical density *vs.* normalized Gaussian density.

From Figure 1.20, we note that market returns have kurtosis higher than that of the normal distribution, i.e. their distribution is *leptokurtic*, in addition to showing some negative skewness.

The next code and graph present a comparison of market prices to a calibrated lognormal distribution using 72 yearly return graphs of the S&P 500 index from 1950 to 2022 as in Figure 1.16.

```
library(quantmod); getSymbols("^GSPC",from="1950-01-01",to="2022-12-31",src="yahoo")
stock<-Cl(`GSPC`); s=0;y=0;j=0;ct=0;N=240;nsim=72; X= matrix(0, nsim, N);
for (i in 1:nrow(GSPC)){if (s==0 && grepl('-01-0',index(stock[i]))) {if (ct==0 || X[y,N]>0)
{y=y+1;j=1;s=1;ct=ct+1;}}
if (j<=N) {X[y,j]=as.numeric(stock[i]);};if (grepl('-02-0',index(stock[i]))) {s=0;};j=j+1;}
t<- 0:(N-1); dt<- 1.0/N; stock=X[,N]/X[,1];stk.dens=density(stock,na.rm=TRUE)
x<- seq(min(stk.dens$x), 1.2*max(stk.dens$x), length=100); dev.new(width=10, height=5)
qx<- dlnorm(x,mean=mean(log(stock)), sd=sd(log(stock)))
plot(x,qx,type="l",lty=2,lwd=3,col="blue",xlab="Prices",ylim=c(0,max(stk.dens$y)),
  ylab="Density", main="", panel.first= abline(h=0,col='grey',lwd=.2))
lines(stk.dens,xlab='x',lwd=3,col="red",ylab='',main='',las=1,cex.axis=1,cex.lab=1,
  xaxs='i',yaxs='i')
legend("topright",c("Emp. density","Lognormal density"),col=c("red", "blue"),lty=1:2,cex=1.2)
```

Listing 1.33: R code - Empirical *vs.* lognormal distribution.

```
import numpy as np, matplotlib.pyplot as plt, yfinance as yf
from scipy.stats import lognorm, gaussian_kde
gspc= yf.download("^GSPC", start="1950-01-01", end="2022-12-31", progress=False)
stock=gspc['Close'].values;dates=gspc.index;N=240;ns=72;X=np.zeros((ns,N));y=j=ct=0;
[( (d := dates[i]).month == 1 and d.day <= 9 and (ct == 0 or X[y-1,N-1] > 0) and
   (y := y+1) and not (j := 0) and not (ct := ct+1),
   y > 0 and j < N and not X.___setitem___((y-1, j), stock[i].item()) and not (j := j+1) ) for i in
  range(len(stock))]; stk_r=X[:,N-1] / X[:,0];x=np.linspace(stk_r.min(),1.2*stk_r.max(),500)
stk_r=stk_r[stk_r>0];  y_emp= gaussian_kde(stk_r)(x)
log_stock= np.log(stk_r); mu_log=np.mean(log_stock); sigma_log=np.std(log_stock)
y_lognorm= lognorm.pdf(x, s=sigma_log, scale=np.exp(mu_log)); plt.figure(figsize=(10,5))
plt.plot(x, y_lognorm, '--', c='b', lw=3, label='Lognormal density');plt.tight_layout()
plt.plot(x, y_emp, c='r', lw=3, label='Empirical density');plt.legend(loc='upper right')
plt.axhline(0, color='grey', lw=.2); plt.xlabel('Prices'); plt.ylabel('Density')
plt.ylim(0, 1.1*max(y_emp.max(), y_lognorm.max()));plt.grid(True, alpha=.3);plt.show()
```

Listing 1.34: Python **3.8** code - Empirical *vs.* lognormal distribution.

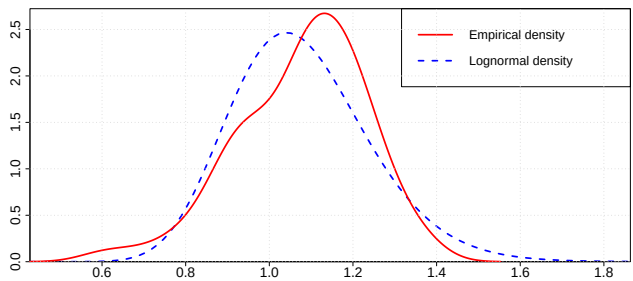


Fig. 1.21: Empirical density *vs.* normalized lognormal density.

Power tail distributions

We note that the empirical density has significantly higher kurtosis and non zero skewness in comparison with the Gaussian probability density.

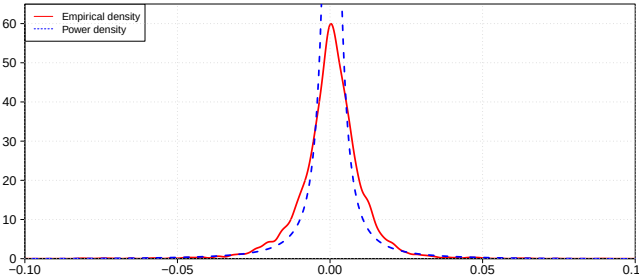


Fig. 1.22: Empirical density *vs.* power tail density.

On the other hand, power tail probability densities of the form $\varphi(x) \simeq C_\alpha/|x|^\alpha$, $|x| \to \infty$, can provide a better fit of empirical probability density functions, as shown in Figure 1.22. The fit of empirical probability density function displayed in Figure 1.22 uses a power probability density function defined by a rational fraction as in the following script.

```
install.packages("pracma"); library(pracma); x<- seq(-.25,.25, length=1000)
stk.dens=density(stk.rtn,na.rm=TRUE, from=-.1, to=.1, n= 1000)
a<-rationalfit(stk.dens$x,stk.dens$y, d1=2, d2=2); dev.new(width=16,height=8)
plot(stk.dens$x,stk.dens$y, lwd=4, type= "l",xlab= '', col="red",ylab= '', main= '',
  xlim=c(-.1,.1), ylim=c(0,65), xaxs="i", yaxs="i", las=1, cex.lab=1.8, cex.axis=1.8)
lines(x,(a$p1[3]+a$p1[2]*x+a$p1[1]*x^2)/(a$p2[3]+a$p2[2]*x+a$p2[1]*x^2),
  type="l",lty=2,col="blue",xlab="x value",lwd=4, ylab="Density",main=""); grid(lwd= 2)
legend("topleft",c("Empirical density", "Power density"),col=c("red", "blue"), lty=1:2, cex=1.5)
```

Listing 1.35: R code - Power tail fitting.

The output of the above command is command is

```
$p1
[1] -0.184717249 -0.001591433  0.001385017

$p2
[1]  1.000000e+00 -6.460948e-04  1.314672e-05
```

which yields a rational fraction of the form

$$x \longmapsto \frac{0.001385017 - 0.001591433 \times x - 0.184717249 \times x^2}{1.314672\ 10^{-5} - 6.460948\ 10^{-4} \times x + x^2}$$
$$\simeq -0.184717249 - \frac{0.001591433}{x} + \frac{0.001385017}{x^2},$$

which approximates the empirical probability density function of DJI returns in the least squares sense.*.

```
import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import gaussian_kde;
from scipy.optimize import curve_fit; x=np.linspace(-.25,.25,1000);
stock_d=gaussian_kde(returns_values); x_d=np.linspace(-.1,.1,1000); y_d=stock_d(x_d);
rational_func=lambda x,a,b,c,d,e:(a*x**2+b*x+c)/(d*x**2+e*x+1);
popt,_=curve_fit(rational_func,x_d,y_d,maxfev=5000);
plt.figure(figsize=(16,8)); plt.plot(x_d,y_d,lw=4,c='r');plt.ylim(0,65);plt.grid(lw=2)
plt.plot(x,rational_func(x,*popt),ls='--',c='b',lw=4); plt.xlim(-.1,.1)
plt.legend(['Emp.-density','Rational-approximation'],loc='upper left',fontsize=15);plt.show()
```

Listing 1.36: Python code - Power tail fitting.

A possible solution to this tail problem is to use stochastic processes with jumps, which can account for sudden variations in asset prices. On the other hand, such jump models are generally based on the Poisson distribution which has a slower tail decay than the Gaussian distribution. This allows one to assign higher probabilities to extreme events, resulting in a more realistic modeling of asset prices. *Stable distributions* with parameter $\alpha \in (0,2)$ provide typical examples of probability laws with power tails, as their probability density functions behave asymptotically as $x \mapsto C_\alpha/|x|^{1+\alpha}$ when $x \to \pm\infty$.

1.5 Gram–Charlier Expansions

Moments and cumulants of random variables

In this section, we search for a better fit of market return distributions using the additional information provided by higher order moments.

We will use the sequence $(\kappa_n(X))_{n\geqslant 1}$ of cumulants of a random variable X, defined from its Moment Generating Function (MGF)

$$\mathcal{M}_X(t) := \mathbb{E}\big[\mathrm{e}^{tX}\big] = 1 + \sum_{n\geqslant 1} \frac{t^n}{n!}\mathbb{E}[X^n], \tag{1.15}$$

for t in a neighborhood of 0, see Thiele (1899).

Definition 1.10. *The* cumulants *of a random variable X are the coefficients $(\kappa_n(X))_{n\geqslant 1}$ appearing in the series expansion*

$$\log \mathbb{E}\big[\mathrm{e}^{tX}\big] = \sum_{n\geqslant 1} \kappa_n(X)\frac{t^n}{n!}, \tag{1.16}$$

of the logarithmic moment generating function (log-MGF) of X.

* Download the corresponding **IPython notebook**† that can be run **here** or **here**.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

The first cumulants of X can be identified as follows.

a) First moment and cumulant. We have $\kappa_1(X) = \mathbb{E}[X]$.

b) Variance and second cumulant. We have

$$\kappa_2(X) = \mathbb{E}\big[X^2\big] - (\mathbb{E}[X])^2 = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big],$$

and $\sqrt{\kappa_2(X)}$ is the *standard deviation* of X.

c) The third cumulant of X is given as the third central moment

$$\kappa_3(X) = \mathbb{E}[(X - \mathbb{E}[X])^3].$$

d) Similarly, the fourth cumulant of X satisfies

$$\begin{aligned}\kappa_4(X) &= \mathbb{E}[(X - \mathbb{E}[X])^4] - 3(\kappa_2(X))^2 \\ &= \mathbb{E}[(X - \mathbb{E}[X])^4] - 3\big(\mathbb{E}\big[(X - \mathbb{E}[X])^2\big]\big)^2.\end{aligned}$$

Example: Gaussian moments and cumulants

When X is centered we have $\kappa_1^X = 0$ and $\kappa_2^X = \mathbb{E}\big[X^2\big] = \mathrm{Var}\,[X]$, and X becomes Gaussian if and only if $\kappa_n^X = 0$, $n \geqslant 3$, *i.e.*

$$\kappa_n^X = \mathbb{1}_{\{n=2\}}\sigma^2, \qquad n \geqslant 1,$$

or

$$\big(\kappa_1^X, \kappa_2^X, \kappa_3^X, \kappa_4^X, \ldots\big) = (0, \sigma^2, 0, 0, \ldots).$$

Example: Poisson moments and cumulants

In the particular case of a Poisson random variable $Z \simeq \mathcal{P}(\lambda)$ with intensity $\lambda > 0$, we have

$$\begin{aligned}\mathbb{E}_\lambda\big[\mathrm{e}^{tZ}\big] &= \sum_{n\geqslant 0} \mathrm{e}^{nt}\mathbb{P}(Z = n) \\ &= \mathrm{e}^{-\lambda}\sum_{n\geqslant 0}\frac{(\lambda \mathrm{e}^t)^n}{n!} \\ &= \mathrm{e}^{\lambda(\mathrm{e}^t - 1)}, \qquad t \in \mathbb{R}_+, \end{aligned} \tag{1.17}$$

hence $\kappa_n^Z = \lambda$, $n \geqslant 1$, or

$$\big(\kappa_1^Z, \kappa_2^Z, \kappa_3^Z, \kappa_4^Z, \ldots\big) = (\lambda, \lambda, \lambda, \lambda, \ldots),$$

Definition 1.11. *i) The* skewness *of X is defined as*

$$\mathrm{Sk}_X := \frac{\kappa_3(X)}{(\kappa_2(X))^{3/2}} = \frac{\mathbb{E}\big[(X-\mathbb{E}[X])^3\big]}{(\mathbb{E}[(X-\mathbb{E}[X])^2])^{3/2}}.$$

ii) The excess kurtosis *of X is defined as*

$$\mathrm{EK}_X := \frac{\kappa_4(X)}{(\kappa_2(X))^2} = \frac{\mathbb{E}[(X-\mathbb{E}[X])^4]}{(\mathbb{E}[(X-\mathbb{E}[X])^2])^2} - 3.$$

The cumulants of X were originally called "semi-invariants" due to the property $\kappa_n(X+Y) = \kappa_n(X) + \kappa_n(Y)$, $n \geqslant 1$, when X and Y are independent random variables. Indeed, in this case we have

$$\begin{aligned}
\sum_{n\geqslant 1} \kappa_n(X+Y)\frac{t^n}{n!} &= \log\left(\mathbb{E}\big[\mathrm{e}^{t(X+Y)}\big]\right) \\
&= \log\left(\mathbb{E}\big[\mathrm{e}^{tX}\big]\mathbb{E}\big[\mathrm{e}^{tY}\big]\right) \\
&= \log\mathbb{E}\big[\mathrm{e}^{tX}\big] + \log\mathbb{E}\big[\mathrm{e}^{tY}\big] \\
&= \sum_{n\geqslant 1} \kappa_n(X)\frac{t^n}{n!} + \sum_{n\geqslant 1} \kappa_n(Y)\frac{t^n}{n!} \\
&= \sum_{n\geqslant 1} \big(\kappa_n(X)+\kappa_n(Y)\big)\frac{t^n}{n!},
\end{aligned}$$

showing that $\kappa_n(X+Y) = \kappa_n(X) + \kappa_n(Y)$, $n \geqslant 1$. Let now

$$\varphi(x) := \frac{1}{\sqrt{2\pi}}\mathrm{e}^{-x^2/2}, \qquad x \in \mathbb{R},$$

denote the standard normal density function, and let

$$\Phi(x) := \int_{-\infty}^{x} \varphi(y)dy, \qquad x \in \mathbb{R},$$

denote the standard normal cumulative distribution function. Let also

$$H_n(x) := \frac{(-1)^n}{\varphi(x)}\frac{\partial^n \varphi}{\partial x^n}(x), \qquad x \in \mathbb{R},$$

denote the Hermite polynomial of degree n, with $H_0(x) = 1$. Proposition 1.12 summarizes the Gram–Charlier expansion method to obtain series expansion of a probability density function, see Gram (1883), Charlier (1914) and § 17.6 of Cramér (1946).

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

Proposition 1.12. *(Proposition 2.1 in Tanaka et al. (2010)) The Gram–Charlier expansion of the continuous probability density function $\phi_X(x)$ of a random variable X is given by*

$$\phi_X(x) = \frac{1}{\sqrt{\kappa_2(X)}}\varphi\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right) + \frac{1}{\sqrt{\kappa_2(X)}}\varphi\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right)\sum_{n=3}^{\infty} c_n H_n\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right),$$

where $c_0 = 1$, $c_1 = c_2 = 0$, and the sequence $(c_n)_{n\geqslant 3}$ is given from the cumulants $(\kappa_n(X))_{n\geqslant 1}$ of X as

$$c_n = \frac{1}{(\kappa_2(X))^{n/2}} \sum_{m=1}^{[n/3]} \sum_{\substack{l_1+\cdots+l_m=n \\ l_1,\ldots,l_m\geqslant 3}} \frac{\kappa_{l_1}(X)\cdots\kappa_{l_m}(X)}{m!l_1!\cdots l_m!}, \qquad n \geqslant 3.$$

The coefficients c_3 and c_4 can be expressed from the skewness $\kappa_3(X)/(\kappa_2(X))^{3/2}$ and the excess kurtosis $\kappa_4(X)/(\kappa_2(X))^2$ as

$$c_3 = \frac{\kappa_3(X)}{3!(\kappa_2(X))^{3/2}} = \frac{\mathrm{Sk}_X}{3!} \quad \text{and} \quad c_4 = \frac{\kappa_4(X)}{4!(\kappa_2(X))^2} = \frac{\mathrm{EK}_X}{4!}.$$

a) The second-order expansion

$$\phi_X^{(1)}(x) = \frac{1}{\sqrt{\kappa_2(X)}}\varphi\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right)$$

corresponds to normal moment matching approximation.

b) The third-order expansion is given by

$$\phi_X^{(3)}(x) = \frac{1}{\sqrt{\kappa_2(X)}}\left(1 + c_3 H_3\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right)\right)\varphi\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right).$$

c) The fourth-order expansion is given by

$$\phi_X^{(4)}(x) = \frac{1}{\sqrt{\kappa_2(X)}}\left(1 + c_3 H_3\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right) + c_4 H_4\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right)\right)\varphi\left(\frac{x-\kappa_1(X)}{\sqrt{\kappa_2(X)}}\right).$$

```
library(quantmod); getSymbols("^DJI", from= "1990-01-03", to= Sys.Date(), src= "yahoo")
stock=Ad(DJI); stock_rtn=diff(log(stock)); data=as.vector(na.omit(stock_rtn))
m=mean(data); s=sd(data); dens=density(data); dev.new(width= 16, height= 8)
x=seq(-.25,.25, length.out= 1000); y_emp=approx(dens$x, dens$y, xout= x)$y
y_gauss=dnorm(x, mean= m, sd= s); mu3=mean((data - m)^3); mu4=mean((data - m)^4)
z=(x-m)/s;c3=mu3/(6*s^3);c4=(mu4-3*s^4)/(24*s^4);g<-dnorm(x,m,s)
d3=g*(1+c3*(z^3-3*z)); d4=g*(1+c3*(z^3-3*z)+c4*(z^4-6*z^2+3))
plot(x, y_emp, type= 'l', lwd= 4, col= "red", xlab= 'x', ylab= 'Density', xlim= c(-.1,.1),
  ylim=c(0, 65), cex.lab= 1.8, cex.axis= 1.8)
lines(x, y_gauss, lty= 2, lwd= 4, col= "blue"); lines(x, d3, lty= 2, lwd= 4, col= "green")
lines(x, d4, lty= 2, lwd= 4, col= "purple"); grid(lwd= 2)
legend("topleft",c("Empirical density","Gaussian density", "3rd order Gram-Charlier","4th order
  Gram-Charlier"),col=c("red","blue","green","purple"),lty=c(1,2,2,2),lwd=4,cex=1.2)
```

Listing 1.37: R code - Gram–Charlier expansions.

```
import numpy as np, matplotlib.pyplot as plt, yfinance as yf;
from scipy.stats import gaussian_kde, norm, moment
dji= yf.download("^DJI", start="1990-01-03", progress=False)['Close']
stock_rtn= np.log(dji).diff(); dat= stock_rtn.dropna().values.flatten();
m, s= dat.mean(), dat.std(); dens= gaussian_kde(dat); x= np.linspace(-.25,.25, 1000);
y_emp= dens(x); y_gauss= norm.pdf(x, loc=m, scale=s); c3=moment(dat-m,3)/6/s**3;
z=(x-m)/s;c4=(moment(dat-m,4)-3*s**4)/(24*s**4);d3=norm.pdf(x,m,s)*(1+c3*(z**3 - 3*z))
d4=norm.pdf(x,m,s)*(1+c3*(z**3-3*z)+c4*(z**4-6*z**2+3))
plt.figure(figsize=(16,8)); plt.plot(x, y_emp, 'r', lw=4, label='Empirical density')
plt.plot(x, y_gauss, '--b', lw=4, label='Gaussian density')
plt.plot(x, d3, '--g', lw=4, label='3rd order Gram-Charlier')
plt.plot(x, d4, '--', c='m', lw=4, label='4th order Gram-Charlier');plt.tight_layout()
plt.xlim(-.1,.1);plt.ylim(0,65);plt.xlabel('x',fontsize=18);plt.ylabel('Density',fontsize=18)
plt.legend(fontsize=16);plt.grid(True,lw=2);plt.tick_params(axis='both',labelsize=16);plt.show()
```

Listing 1.38: Python code - Gram–Charlier expansions.

Edgeworth expansions can be similarly generated using the R package SimMultiCorrData as in the following R script.

```
install.packages("SimMultiCorrData");install.packages("PDQutils")
library(SimMultiCorrData);library(PDQutils); library(quantmod)
getSymbols("^DJI",from="1990-01-03",to=Sys.Date(),src="yahoo");stock=Ad(`DJI`);
stk.rtn=diff(log(stock)); returns<- as.vector(stk.rtn);stk.dens=density(stk.rtn,na.rm=TRUE)
dev.new(width=16,height=8);m<-calc_moments(returns[!is.na(returns)]);
x<- stk.dens$x; qx<- dnorm(x,mean=m[1],sd=m[2])
plot(x,stk.dens$y, xlab= 'x', type= 'l', lwd=4, col="red",ylab= '', main= '', xlim= c(-.1,.1),
  ylim=c(0,65), xaxs="i", yaxs="i", las=1, cex.lab=1.8, cex.axis=1.8)
grid(lwd= 2); cumulants<-c(m[1],m[2]**2); d2<- dapx_edgeworth(x, cumulants)
lines(x, d2, type="l", lty=2, lwd=4, col="blue");cumulants<-c(m[1],m[2]**2,m[3]*m[2]**3)
d3<- dapx_edgeworth(x, cumulants);lines(x, d3, type="l", lty=2, lwd=4, col="green")
cumulants<-c(m[1],m[2]**2,.5*m[3]*m[2]**3,.2*m[4]*m[2]**4)
d4<- dapx_edgeworth(x, cumulants);lines(x, d4, type="l", lty=2, lwd=4, col="purple")
legend("topleft",c("Empirical density", "Gaussian density", "3rd order Gram-Charlier", "4th
  order Gram-Charlier"),col=c("red", "blue", "green", "purple"), lty=1:2,cex=1.5)
```

Listing 1.39: R code - Edgeworth expansions.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

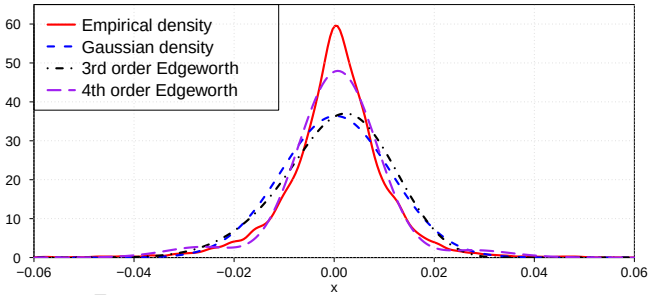


Fig. 1.23: Edgeworth expansions up to the fourth order.

Exercises

Exercise 1.1 Let $c > 0$. Using the definition of Brownian motion $(B_t)_{t\in\mathbb{R}_+}$, show that:

a) $(B_{c+t} - B_c)_{t\in\mathbb{R}_+}$ is a Brownian motion.
b) $(cB_{t/c^2})_{t\in\mathbb{R}_+}$ is a Brownian motion.

Exercise 1.2 Solve the stochastic differential equation

$$dS_t = \mu S_t dt + \sigma S_t dB_t, \tag{1.18}$$

defining geometric Brownian motion $(S_t)_{t\in\mathbb{R}_+}$, where $\mu, \sigma \in \mathbb{R}$.

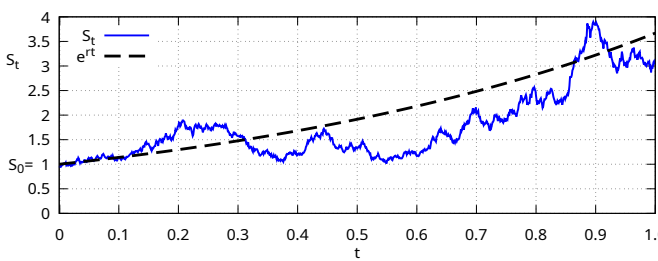


Fig. 1.24: Sample path of (1.18) with $r = 1$ and $\sigma^2 = 0.5$.

Exercise 1.3

a) Using the Gaussian moment generating function (MGF) formula (A.41), compute the n-th order moment $\mathbb{E}[S_t^n]$ for all $n \geqslant 1$.
b) Compute the lognormal mean and variance

$$\mathbb{E}[S_t] = S_0 e^{rt} \quad \text{and} \quad \mathrm{Var}[S_t] = S_0^2 e^{2rt}\big(e^{\sigma^2 t} - 1\big), \quad t \geqslant 0.$$

Exercise 1.4 Consider two assets whose prices $S_t^{(1)}$, $S_t^{(2)}$ at time $t \in [0,T]$ follow the geometric Brownian dynamics

$$dS_t^{(1)} = \mu S_t^{(1)} dt + \sigma_1 S_t^{(1)} dW_t^{(1)} \quad \text{and} \quad dS_t^{(2)} = \mu S_t^{(2)} dt + \sigma_2 S_t^{(2)} dW_t^{(2)},$$

$t \in [0,T]$, where $\big(W_t^{(1)}\big)_{t\in[0,T]}$, $\big(W_t^{(2)}\big)_{t\in[0,T]}$ are two Brownian motions with correlation $\rho \in [-1,1]$, *i.e.* we have $\mathbb{E}\big[W_t^{(1)} W_t^{(2)}\big] = \rho t$. Compute $\mathrm{Var}\big[S_t^{(2)} - S_t^{(1)}\big]$, $t \in [0,T]$.

Exercise 1.5 We consider an economy in which individual income is modeled over time by a geometric Brownian motion

$$S_t = S_0 e^{\sigma B_t + \mu t - \sigma^2 t/2}, \qquad t \geqslant 0.$$

Compute the value

$$\mathrm{Theil}_t := \log \mathbb{E}[S_t] - \mathbb{E}[\log S_t]$$

of the Theil (1967) inequality index at time $t \geqslant 0$ in this economy.

Hint: You may use the moment generating function (13.60) of the normal distribution.

Exercise 1.6 The statistics of 74 yearly return graphs of the S&P 500 index from 1950 to 2024 show that the *yearly* returns have mean 0.0949535 and standard deviation 0.1663085.

```
library(quantmod); getSymbols("^GSPC",from="1950-01-01",to="2024-12-31",src="yahoo")
index<-Cl(`GSPC`); returns<- periodReturn(index, period= "yearly", type= "arithmetic")
barplot(as.numeric(returns), names.arg= format(index(returns), "%Y"), col=
 ifelse(as.numeric(returns) > 0, "blue", "red"), las= 2, main= "Yearly Returns of S&P 500",
 ylab= "Yearly Log Returns", xlab= "Year"); mean(returns); sd(returns)
```

Listing 1.40: R code - Mean and standard deviation estimates.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html

```
import yfinance as yf; import matplotlib.pyplot as plt; import numpy as np
gspc = yf.download("^GSPC", start="1950-01-01", end="2024-12-31", progress=False)
index = gspc['Close']; returns = index.resample('YE').last().pct_change().dropna().squeeze()
val = returns.values; yrs = returns.index.year; plt.figure(figsize=(12, 6))
plt.bar(range(len(val)), [float(x) for x in val], color=['b' if float(x) > 0 else 'r' for x in val])
plt.xticks(range(len(val)),[str(y) for y in yrs],rotation=90);plt.tight_layout();plt.xlabel("Year")
plt.title("Yearly Returns of S&P 500");plt.ylabel("Yearly Arithmetic Returns");plt.show()
print(f"Mean return: {returns.mean():.4f}"); print(f"Standard deviation: {returns.std():.4f}")
```

Listing 1.41: Python code - Mean and standard deviation estimates.

Using (1.13), estimate the values of μ and σ from the above data.

Exercise 1.7 Consider

- a risky asset whose price $(S_t)_{t\geqslant 0}$ is modeled by a geometric Brownian motion as
$$dS_t = \mu S_t dt + \sigma S_t dB_t,$$
and
- a riskless asset whose price $(A_t)_{t\geqslant 0}$ is modeled as
$$dA_t = rA_t dt.$$

Our goal is to construct a portfolio strategy $(\xi_t, \eta_t)_{t\geqslant 0}$ that leverages the risky asset with a factor $\beta > 0$, and whose value $V_t = \xi_t S_t + \eta_t A_t$ remains above A_t, $t \geqslant 0$, after starting from $V_0 > A_0$. For this, we assume that:

- $(\xi_t, \eta_t)_{t\geqslant 0}$ is *self-financing*, *i.e.*
$$dV_t = \xi_t dS_t + \eta_t dA_t;$$

- if $V_t > A_t$, then $\xi_t = (V_t - A_t)\beta/S_t$ and $\eta_t := (V_t - \xi_t S_t)/A_t$;
- If $V_t \leqslant A_t$, we let $\xi_t = 0$ and $\eta_t := (V_t - \xi_t S_t)/A_t$.

a) Compute $dV_t - dA_t$, and show that $C_t := V_t - A_t$ is a geometric Brownian motion.
b) Find the value of $V_t = A_t + C_t$, $t \geqslant 0$.

Exercise 1.8 An investor placed a 6-month SGD Time Deposit with ACBC Bank at the interest rate $r = 2.7\%$ per annum for a total value of $A = \$3,000$, and received \$40.83 in interest at maturity. The year was 2023, which has 365 days. We denote by $t \in \mathbb{N}$ the investment duration, counted as an integer number of *days*.

a) Did ACBC bank use
   i) exponential compounding of the form $e^{rt/365}$, or

ii) linear compounding of the form $1 + rt/365$,

for interest calculation?

b) What was the actual duration of investment in integer number of days ?

Exercise 1.9 (Fernholz (2002)) Stochastic portfolio theory. We consider a market with $n \geqslant 1$ stocks whose prices follow the log-return/arithmetic return dynamics

$$d \log X_i(t) = \gamma_i(t)dt + \sum_{k=1}^{n} \sigma_{i,k}(t)dB_k(t),$$

or

$$\frac{dX_i(t)}{X_i(t)} = \mu_i(t)dt + \sum_{k=1}^{n} \sigma_{i,k}(t)dB_k(t), \quad i = 1, \ldots, n,$$

where $B_1(t), \ldots, B_n(t)$ are independent Brownian motions. Assuming that every stock issues only a single share, we consider the market portfolio with value

$$V(t) = X_1(t) + \cdots + X_n(t) = \pi_1(t)V(t) + \cdots + \pi_n(t)V(t), \qquad t \geqslant 0,$$

which contains exactly one share of each stock at any time $t \geqslant 0$, where $\pi_i(t) := X_i(t)/V(t)$, $i = 1, \ldots, n$, denotes the relative market capitalization of stock n^o i at time $t \geqslant 0$.

a) Show that the portfolio's arithmetic returns can be written as

$$\frac{dV(t)}{V(t)} = \bar{\mu}(t)dt + \sum_{k=1}^{n} \bar{\sigma}_k(t)dB_k(t),$$

where $\bar{\mu}(t)$, $\sigma_1(t), \ldots, \sigma_n(t)$ will be computed in terms of $\pi_1(t), \ldots, \pi_n(t)$ and $\mu_1(t), \ldots, \mu_n(t)$, $\sigma_{1,k}(t), \ldots, \sigma_{n,k}(t)$.

b) Using the portfolio log-returns written as

$$d \log V(t) = \bar{\gamma}(t)dt + \sum_{k=1}^{n} \bar{\sigma}_k(t)dB_k(t),$$

compute the log-return dynamics $d \log \pi_i(t)$ of the relative market capitalizations in terms of $\bar{\gamma}(t), \gamma_1(t), \ldots, \gamma_n(t), \bar{\sigma}_1(t), \ldots, \bar{\sigma}_n(t), \sigma_{1,k}(t), \ldots, \sigma_{n,k}(t)$.

This version: July 14, 2026
https://personal.ntu.edu.sg/nprivault/indext.html